Praktikum 1 — Konsep Bahasa Pemrograman C++

1.1 Tujuan Praktikum

- Mahasiswa mampu memahami dasar-dasar bahasa pemrograman C++.
- Mahasiswa mampu mengetahui perbedaan antara bahasa pemrograman C dan C++.
- Mahasiswa mampu menerapkan prinsip-prinsip pemrograman C++.

1.2 Teori Singkat

a. Sejarah singkat

C++ dikembangkan pada awal tahun 1980-an oleh Bjarne Stroustrup sebagai perpanjangan dari bahasa pemrograman C yang disebut *C with Classes*. Rilis komersial pertama adalah pada tahun 1985, dan bahasa terus berkembang dengan standar baru, termasuk C++11, C++14, C++17, dan C++20. Saat ini, C++ adalah bahasa populer untuk mengembangkan aplikasi perangkat lunak berkinerja tinggi, khususnya di industri game, keuangan, dan kedirgantaraan.

b. Struktur dasar program C++

Program C++ terdiri dari satu atau lebih fungsi, dengan fungsi `main()` sebagai titik awal program.

```cpp
// Sertakan file header yang diperlukan
#include <iostream>

// Deklarasikan namespace yang diperlukan
using namespace std;

// Definisikan fungsi utama
int main() {
    // Pernyataan program
    cout << "Hello, world!" << endl;

    // Kembalikan nilai integer ke sistem operasi
    return 0;
}
```

Bagian-bagian dari struktur dasar:

- `#include` : menyertakan file header, yaitu kumpulan fungsi dan variabel yang telah ditentukan sebelumnya. Pada contoh ini kita menyertakan `iostream` yang berisi fungsi untuk operasi input dan output.

- `using namespace std;` : mendeklarasikan namespace standar (`std`) yang berisi banyak fungsi dan objek berguna.
- `int main() { ... }` : fungsi utama program. Pada contoh ini, `cout` (bagian dari `namespace std`) mencetak `"Hello, world!"` ke konsol, dan `endl` mencetak karakter baris baru.
- `return 0;` : mengembalikan nilai integer ke sistem operasi. Nilai `0` berarti program berhasil dieksekusi, nilai bukan nol berarti terjadi kesalahan.

> **Catatan:** urutan direktif `#include` dan pernyataan `using namespace` dapat bervariasi tergantung kebutuhan program.

c. Variabel dan tipe data

Variabel digunakan untuk menyimpan dan memanipulasi data. Variabel dideklarasikan bersama tipe datanya, yang menentukan jenis data yang dapat disimpan. Tipe data bawaan C++ antara lain:

Tipe	Keterangan	Contoh
`int`	bilangan bulat	`int nomorku = 42;`
`double`	bilangan desimal (floating-point)	`double myDouble = 3.14159;`
`bool`	Boolean, bernilai `true` atau `false`	`bool myBoolean = true;`
`char`	satu karakter	`char myChar = 'a';`
`string`	rangkaian karakter	`string myString = "Halo, dunia!";`

Sintaks deklarasi variabel:

```
<tipeData> <namaVariabel> = <nilai>;
```

> **Catatan:** bilangan desimal ditulis dengan **titik** (`3.14159`), bukan koma. Tipe `string` memerlukan `#include <string>` .

```
int myNumber = 42;
double myDouble = 3.14159;
bool myBoolean = true;
char myChar = 'a';
string myString = "Hello, world!";
```

d. Operasi input / output

Output menggunakan objek `cout` (bagian dari `namespace std`) bersama operator `<<` :

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, world!" << endl;
    return 0;
}
```

Operator `<<` menyisipkan string ke objek `cout`, yang kemudian mencetaknya ke konsol. `endl` mencetak baris baru.

Input menggunakan objek `cin` dan operator `>>`. Contoh berikut meminta pengguna memasukkan sebuah angka, membacanya, lalu mencetaknya kembali:

```cpp
#include <iostream>
using namespace std;

int main() {
    int myNumber;
    cout << "Enter a number: ";
    cin >> myNumber;
    cout << "You entered: " << myNumber << endl;
    return 0;
}
```

Operator `>>` mengekstrak masukan pengguna dari objek `cin` dan menyimpannya ke variabel `myNumber`.

> **Catatan:** `cin >> variabelString` hanya membaca **satu kata** (berhenti pada spasi). Untuk membaca satu baris utuh gunakan `getline(cin, variabelString);`.

Input/output file menggunakan library `fstream`. Contoh berikut menulis string ke file `output.txt`:

```cpp
#include <fstream>
using namespace std;

int main() {
    ofstream outfile("output.txt");
    outfile << "Hello, file!";
    outfile.close();
    return 0;
}
```

Objek `ofstream` membuka file `output.txt` untuk ditulis, operator `<<` menyisipkan string ke file, dan `close()` menutup file setelah selesai. Untuk membaca file gunakan `ifstream`.

e. Struktur kontrol (if-else, switch-case, perulangan)

Struktur kontrol mengatur alur eksekusi program berdasarkan kondisi atau perulangan.

i. If-else

```
if (condition) {
    // kode jika kondisi benar
} else {
    // kode jika kondisi salah
}
```

Contoh: menampilkan "positive" atau "non-positive" bergantung pada nilai `x`:

```
if (x > 0) {
    cout << "positive" << endl;
} else {
    cout << "non-positive" << endl;
}
```

ii. Switch-case

`switch-case` mirip dengan `if-else`, tetapi mengeksekusi kode berbeda berdasarkan nilai suatu variabel atau ekspresi.

```
switch (expression) {
    case value1:
        // kode jika expression bernilai value1
        break;
    case value2:
        // kode jika expression bernilai value2
        break;
    // case tambahan ...
    default:
        // kode jika tidak ada case yang cocok
}
```

Contoh: mencetak nama bulan berdasarkan `monthNum`:

```
switch (monthNum) {
    case 1:
        cout << "January" << endl;
        break;
    case 2:
        cout << "February" << endl;
        break;
    // case tambahan ...
    default:
        cout << "Invalid month" << endl;
}
```

iii. Perulangan

Perulangan (*loop*) mengeksekusi blok kode berulang kali berdasarkan kondisi tertentu. Ada tiga jenis: `for`, `while`, dan `do-while`.

For loop dipakai ketika jumlah iterasi diketahui sebelumnya:

```
for (initialization; condition; increment/decrement) {
    // kode yang dieksekusi tiap iterasi
}
```

Contoh: mencetak angka 1 sampai 10.

```
for (int i = 1; i <= 10; i++) {
    cout << i << endl;
}
```

While loop dipakai ketika jumlah iterasi tidak diketahui, tetapi loop harus berlanjut selama kondisi terpenuhi:

```
while (condition) {
    // kode yang dieksekusi tiap iterasi
}
```

Contoh: mencetak 10 bilangan genap pertama.

```
int i = 1;
while (i <= 20) {
    if (i % 2 == 0) {
        cout << i << endl;
    }
    i++;
}
```

Do-while loop mirip dengan `while`, tetapi dijalankan **setidaknya satu kali**, bahkan jika kondisi awalnya salah:

```
do {
    // kode yang dieksekusi tiap iterasi
} while (condition);
```

Contoh: terus meminta angka positif sampai pengguna memasukkan angka yang lebih dari 0.

```
int num;
do {
    cout << "Enter a positive number: ";
    cin >> num;
} while (num <= 0);
```

f. Fungsi

Fungsi adalah blok kode yang melakukan tugas tertentu. Fungsi menyederhanakan kode, membuatnya lebih modular dan dapat digunakan kembali.

```
return_type function_name(parameters) {
    // isi fungsi
    return value;
}
```

`return_type` menentukan jenis nilai yang dikembalikan, `function_name` adalah nama fungsi, dan `parameters` adalah daftar parameter input yang dipisahkan koma.

g. Perbandingan C dan C++

C dan C++ adalah bahasa yang berkaitan erat karena C++ merupakan pengembangan dari C. Meskipun demikian terdapat beberapa perbedaan:

Aspek	C	C++
Paradigma	prosedural	prosedural **dan** berorientasi objek (OOP): `class`, *inheritance*, *encapsulation*, *polymorphism*
Kompatibilitas	—	sebagian besar kode C dapat dikompilasi oleh kompiler C++, tetapi tidak sebaliknya
Pengelolaan memori	manual (`malloc` / `free`)	tambahan konstruktor, destruktor, dan konsep RAII (*Resource Acquisition Is Initialization*)
Keterbacaan	lebih sederhana	lebih kompleks karena memiliki lebih banyak fitur
Performa	cenderung sedikit lebih cepat dan ringkas	perbedaannya tidak signifikan pada banyak kasus

C++ menambahkan banyak fitur ke C, terutama untuk mendukung pemrograman berorientasi objek dan pengelolaan memori yang lebih baik. Namun hal ini membuat C++ lebih kompleks bagi pemula, sedangkan

C lebih sederhana dan ringkas untuk proyek yang membutuhkan performa tinggi.

h. Prinsip-prinsip pemrograman C++

1) Menulis kode yang jelas dan ringkas

- *Gunakan nama yang bermakna* untuk variabel, fungsi, dan kelas. Hindari nama satu huruf atau nama umum seperti `temp`, `data`, atau `result`. Gunakan `camelCase` atau `snake_case`.
- *Tulis fungsi dan kelas yang pendek.* Satu fungsi sebaiknya memiliki tujuan yang jelas dan cukup dibaca dalam satu layar.
- *Gunakan komentar dengan bijak.* Komentar menjelaskan **mengapa** kode ditulis seperti itu (algoritma, asumsi, batasan, *trade-off*), bukan **apa** yang dilakukan setiap baris.
- *Buat kode yang mendokumentasikan dirinya sendiri* lewat nama deskriptif, indentasi, dan spasi kosong yang bermakna.
- *Hindari duplikasi kode.* Kumpulkan kode yang sama ke dalam fungsi atau kelas.
- *Gunakan pesan kesalahan yang bermakna:* apa yang salah, di mana, dan bagaimana memperbaikinya.
- *Ikuti gaya yang konsisten* (penamaan, indentasi, spasi, format) di seluruh basis kode.

2) Menghindari kesalahan pemrograman umum

Kesalahan	Penjelasan	Cara menghindari
Kebocoran memori (*memory leak*)	memori dialokasikan secara dinamis tetapi tidak dibebaskan	pasangkan setiap `new` dengan `delete`
Null pointer	dereferensi pointer yang tidak menunjuk alamat valid dapat membuat program crash	inisialisasi pointer dengan alamat valid atau `nullptr`, dan periksa sebelum dereferensi
Akses di luar batas	membaca/menulis di luar batas array menyebabkan perilaku tak terduga	pastikan indeks dalam batas; gunakan `std::vector::at` agar kesalahan terdeteksi saat *runtime*
Integer overflow	nilai terlalu besar untuk tipe data	pilih tipe yang sesuai (`long long`, dsb.) atau pustaka BigInt
Pembagian dengan nol	menyebabkan crash atau perilaku tidak terdefinisi	periksa pembagi sebelum melakukan pembagian
Kebocoran sumber daya	file/soket tidak dilepas setelah dipakai	tutup dengan `.close()` atau bungkus dalam kelas yang menerapkan RAII

3) Praktik terbaik penamaan variabel dan fungsi

1. **Gunakan nama deskriptif dan bermakna** yang menggambarkan tujuan variabel atau fungsi.
2. **Ikuti konvensi penamaan yang konsisten**, misalnya `camelCase`, `PascalCase`, atau `snake_case`.
3. **Gunakan awalan tipe data bila membantu**, misalnya `str` untuk string atau `i` untuk integer.
4. **Gunakan kata kerja untuk nama fungsi**, misalnya `hitungAkarKuadrat()`.
5. **Hindari singkatan dan akronim** kecuali sudah umum di bidang Anda.

1.3 Tugas Pendahuluan

Kerjakan **sebelum** praktikum dimulai (tulis tangan atau ketik) dan kumpulkan di awal sesi.

1. Apa fungsi `#include <iostream>` dan `using namespace std;` pada program C++? Mengapa fungsi `main()` selalu mengembalikan nilai (`return 0;`)?
2. Sebutkan lima tipe data bawaan C++ beserta satu contoh deklarasi dan nilai untuk masing-masing.
3. Jelaskan perbedaan `for` , `while` , dan `do-while` . Pada kondisi apa `do-while` lebih tepat dipakai?
4. Tuliskan fungsi `int kuadrat(int x)` yang mengembalikan kuadrat dari `x` , lalu tunjukkan cara memanggilnya dari `main()` .
5. Sebutkan tiga perbedaan antara bahasa C dan C++, serta tiga kesalahan pemrograman umum beserta cara menghindarinya.

1.4 Pelaksanaan Praktikum

a. Membuka Code::Blocks

1. Buka Code::Blocks.
2. Klik **File → New → Project**.
3. Pilih **Console Application**, klik **Go**.
4. Pilih **C++**, klik **Next**.
5. Beri judul project dan tentukan lokasi penyimpanan.
6. Klik **Next**, lalu **Finish**.
7. File `main.cpp` akan tampil seperti berikut:

```cpp
#include <iostream>

using namespace std;

int main()
{
    cout << "Hello world!" << endl;
    return 0;
}
```

b. Program sederhana: penjumlahan, pengurangan, perkalian, dan pembagian dua angka

Langkah 1 — Ambil nilai dari pengguna. Deklarasikan dua variabel lalu baca nilainya dengan `cin` .

```cpp
int angkaPertama, angkaKedua;
cout << "Masukkan Angka Pertama : ";
cin >> angkaPertama;
cout << "Masukkan Angka Kedua : ";
cin >> angkaKedua;
```

Langkah 2 — Buat fungsi untuk perhitungan matematika.

```
int penjumlahan(int a, int b) {
    return a + b;
}
int pengurangan(int a, int b) {
    return a - b;
}
int perkalian(int a, int b) {
    return a * b;
}
int pembagian(int a, int b) {
    return a / b;
}
```

Langkah 3 — Panggil fungsi pada `main()` untuk mendapatkan hasilnya.

Langkah 4 — Kode lengkap. Pada versi lengkap, pembagian hanya dijalankan jika pembagi bukan nol (lihat *pembagian dengan nol* pada bagian 1.2.h).

```cpp
#include <iostream>
using namespace std;

int penjumlahan(int a, int b) {
    return a + b;
}

int pengurangan(int a, int b) {
    return a - b;
}

int perkalian(int a, int b) {
    return a * b;
}

int pembagian(int a, int b) {
    return a / b;
}

int main() {
    int angkaPertama, angkaKedua;
    cout << "Masukkan Angka Pertama : ";
    cin >> angkaPertama;
    cout << "Masukkan Angka Kedua : ";
    cin >> angkaKedua;

    cout << "Hasil penjumlahan : " << penjumlahan(angkaPertama, angkaKedua) << endl;
    cout << "Hasil pengurangan : " << pengurangan(angkaPertama, angkaKedua) << endl;
    cout << "Hasil perkalian   : " << perkalian(angkaPertama, angkaKedua) << endl;

    if (angkaKedua != 0) {
        cout << "Hasil pembagian   : " << pembagian(angkaPertama, angkaKedua) << endl;
    } else {
        cout << "Pembagian tidak dapat dilakukan (pembagi = 0)" << endl;
    }

    return 0;
}
```

Contoh keluaran untuk masukan `10` dan `3`:

```
Masukkan Angka Pertama : 10
Masukkan Angka Kedua : 3
Hasil penjumlahan : 13
Hasil pengurangan : 7
Hasil perkalian   : 30
Hasil pembagian   : 3
```

> Karena kedua operan bertipe `int`, hasil pembagian `10 / 3` adalah `3` (bagian desimal dibuang).

1.5 Latihan Soal

1. Bilangan prima. Buatlah program yang meminta input suatu bilangan, kemudian menampilkan apakah bilangan tersebut prima atau tidak. Apabila bukan prima, tampilkan juga angka yang dapat membagi habis bilangan tersebut.

Input	Output
`7`	`7 adalah bilangan prima`
`10`	`10 bukan prima, karena dapat dibagi habis 1, 2, 5, 10`
`20`	`20 bukan prima, karena dapat dibagi habis 1, 2, 5, 10, 20`

2. Perkalian dan pembagian tanpa operator `*` dan `/`. Program meminta 2 inputan angka. Buatlah operasi matematika yang melakukan perkalian dan pembagian pada kedua bilangan tersebut **tanpa menggunakan simbol `*` dan `/`**.

Input	Output
`5, 2`	`Perkalian : 10` `Pembagian : 2`
`10, 2`	`Perkalian : 20` `Pembagian : 5`

3. Faktor bilangan. Buatlah program yang meminta input sebuah angka, kemudian carilah faktor dari bilangan tersebut.

Input	Output
`5`	`1, 5`
`20`	`1, 2, 5, 10, 20`

4. Pengurutan. Buatlah program yang meminta input 5 angka, kemudian urutkan angka tersebut dari bilangan terbesar menuju terkecil.

Input	Output
`1, 5, 2, 8, 9`	`9, 8, 5, 2, 1`
`8, 1, 10, 15, 4`	`15, 10, 8, 4, 1`

1.6 Tugas Rumah

Andri adalah anak kelas 2 SD. Ia memiliki tugas untuk menghitung banyaknya kata yang terdapat dalam suatu kalimat. Bantulah Andri dengan membuatkan program yang dapat menghitung banyaknya kata pada sebuah kalimat.

Input	Output
`"Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor"`	`12`
`"Apa kabar"`	`2`

> **Petunjuk:** gunakan `getline(cin, kalimat)` agar spasi ikut terbaca, lalu hitung kata berdasarkan pergantian dari spasi ke karakter bukan spasi.

1.7 Studi Kasus (*Hard Thinking*)

Kembalian Kasir dengan Pecahan Terbatas

Sebuah minimarket ingin program kasir yang menghitung **kembalian dengan jumlah lembar/koin paling sedikit**. Pecahan yang tersedia: Rp100.000, 50.000, 20.000, 10.000, 5.000, 2.000, 1.000, 500, 200, dan 100.

Kebutuhan:

- Input: total belanja dan uang yang dibayarkan (bisa berupa angka tidak valid, negatif, atau kurang dari total).
- Output: jumlah tiap pecahan untuk kembalian.
- Kasus lanjutan: laci kasir hanya memiliki **stok terbatas** tiap pecahan (misalnya Rp2.000 hanya 1 lembar, Rp5.000 tidak ada).

Poin keputusan desain — bagian inilah yang biasanya membuat solusi tiap orang berbeda. Jawab dengan alasan:

1. Cara menyimpan daftar pecahan: banyak `if`, array, atau fungsi terpisah? Mana yang paling mudah diubah bila ada pecahan baru?
2. Algoritma *greedy* (ambil pecahan terbesar dulu) selalu optimal untuk daftar pecahan di atas. Apakah masih optimal bila stok terbatas? Beri contoh angka yang membuktikannya.
3. Bagaimana program bersikap bila kembalian **tidak mungkin** dibayar dengan stok yang ada?
4. Di mana validasi input ditempatkan (di `main`, di fungsi sendiri, atau berulang dengan `do-while`)?
5. Pendekatan iteratif atau rekursif: mana yang lebih jelas dibaca dan mana yang lebih mudah diuji?

Ketentuan pengerjaan:

1. Kerjakan studi kasus di atas dengan solusi **Anda sendiri**. Tidak ada satu jawaban benar; yang dinilai adalah kualitas penalaran dan kebenaran program.

2. Sertakan **laporan singkat (maksimal 1 halaman)** yang menjawab seluruh pertanyaan analisis dan menjelaskan alasan Anda memilih rancangan tersebut dibanding alternatif lain.
3. Uji program dengan **kasus tepi** dan lampirkan minimal 5 kasus uji beserta hasilnya.
4. Diskusikan solusi Anda dengan teman: bandingkan perbedaan rancangan, lalu catat kelebihan dan kekurangannya.

Kriteria penilaian: ketepatan hasil (30%), kerapian dan kejelasan struktur kode (20%), penanganan kasus tepi (20%), kualitas analisis pada laporan (30%).